

Bayesian Neural Networks

From Bayes' Rule to Surrogates

Krishna Kumar

University of Texas at Austin

Roadmap

- 1 Anchor Problem: Noisy Oscillator
- 2 Probability Distributions Primer
- 3 Bayes' Theorem
- 4 Bayesian Linear Regression
- 5 Model Selection via Evidence
- 6 Variational Inference
- 7 Bayesian Neural Networks
- 8 Results: Interpolation vs Extrapolation
- 9 Practical Considerations
- 10 Summary

Why Bayesian?

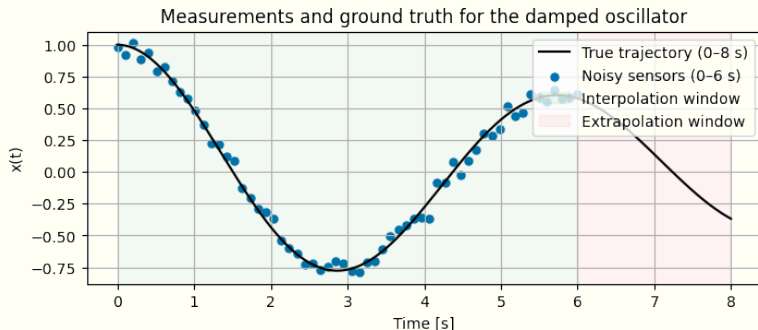
Standard neural networks give point predictions. But in science and engineering we need:

- **Uncertainty quantification:** How confident is the model?
- **Extrapolation warnings:** When is the model outside its training domain?
- **Parameter inference:** What physical parameters are consistent with data?

Bayesian methods provide a principled framework for all three.

Start with a concrete question

- Lightly damped oscillator with noisy sensors on $x(t)$.
- Governing ODE: $\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = 0$
- Unknown: damping ratio ζ , natural frequency ω_n .
- Goal: credible predictions beyond the observed window.



The challenge

- We observe noisy measurements $y_i = x(t_i) + \epsilon_i$ for $t \in [0, 6]$ s.
- We want predictions for $t \in [6, 8]$ s (extrapolation).
- **Key question:** How uncertain should we be about extrapolated predictions?

A standard NN would give a single curve with no uncertainty.

A Bayesian approach gives a *distribution* over predictions.

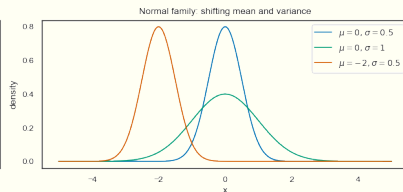
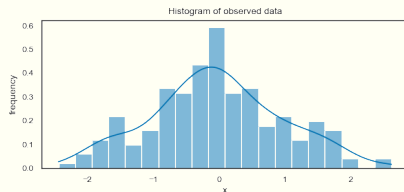
Building blocks: probability distributions

Bayesian models require specifying distributions. Three families cover most needs:

- **Normal** $\mathcal{N}(\mu, \sigma^2)$: measurement noise, weight priors
- **Gamma** $\text{Gamma}(\alpha, \beta)$: positive scales, precisions
- **Uniform** $\text{Uniform}(a, b)$: weakly informative bounded priors

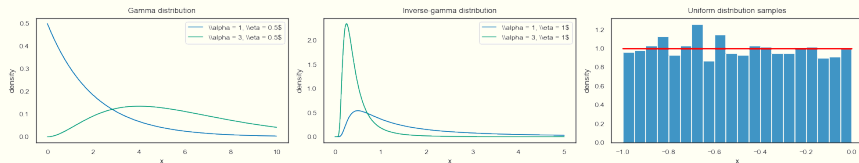
The Normal distribution

$$p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$



- μ shifts the center, σ controls spread.
- Ubiquitous: Central Limit Theorem, maximum entropy for fixed variance.

Gamma and Uniform distributions



- **Gamma:** positive support, models precisions ($\tau = 1/\sigma^2$).
- **Inverse-Gamma:** conjugate prior for variance in Normal likelihood.
- **Uniform:** “I only know the parameter lies in $[a, b]$.”

Bayes' theorem: the update rule

Given data $\mathcal{D}$ and parameters θ :

$$\underbrace{p(\theta|\mathcal{D})}_{\text{posterior}} = \frac{\overbrace{p(\mathcal{D}|\theta)}^{\text{likelihood}} \cdot \overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{evidence}}}$$

- **Prior** $p(\theta)$: what we believe before seeing data.
- **Likelihood** $p(\mathcal{D}|\theta)$: probability of data given parameters.
- **Posterior** $p(\theta|\mathcal{D})$: updated belief after seeing data.
- **Evidence** $p(\mathcal{D})$: normalizing constant (model comparison).

Bayes on the oscillator: setup

Prior beliefs (before seeing data):

- Damping: $\zeta \sim \text{Uniform}(0, 0.25)$ — light to moderate damping
- Frequency: $\omega_n \sim \text{Uniform}(0.6, 1.6)$ rad/s

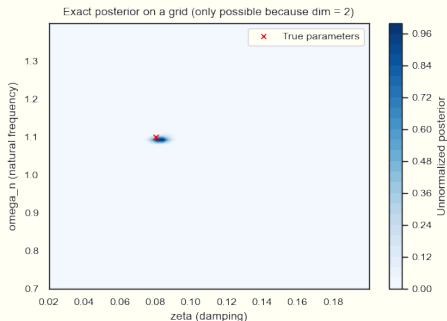
Likelihood (how data relates to parameters):

$$p(\mathcal{D}|\zeta, \omega_n) = \prod_{i=1}^N \mathcal{N}(y_i | x(t_i; \zeta, \omega_n), \sigma^2)$$

where $x(t; \zeta, \omega_n)$ is the ODE solution.

Bayes on the oscillator: posterior

$$p(\zeta, \omega_n | \mathcal{D}) \propto p(\mathcal{D} | \zeta, \omega_n) \cdot p(\zeta) \cdot p(\omega_n)$$



- Grid evaluation feasible because $\dim(\theta) = 2$.
- Posterior concentrates near true values ($\zeta = 0.08, \omega_n = 1.1$).
- Uncertainty reflects noise level and data coverage.

The curse of dimensionality

Grid evaluation worked for 2 parameters. But:

- Neural network with 1000 weights $\Rightarrow 10^{1000}$ grid points needed!
- Even MCMC struggles in high dimensions.

Solution: Variational inference — approximate the posterior with a tractable distribution and optimize.

But first, let's understand exact Bayesian inference in a simpler setting...

Why start with linear regression?

Bayesian linear regression is the **only** case where:

- ➊ Posterior is available in closed form (no approximation needed).
- ➋ Predictive distribution is analytically tractable.
- ➌ We can visualize exactly what “Bayesian” means.

The intuitions carry over to neural networks, where we must approximate.

Model setup

Likelihood: Linear model with Gaussian noise

$$y = \mathbf{w}^T \phi(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \beta^{-1})$$

where $\phi(\mathbf{x}) = [1, \phi_1(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x})]^T$ are basis functions.

Prior: Gaussian on weights

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \alpha^{-1} \mathbf{I})$$

- α = prior precision (large $\alpha \Rightarrow$ weights close to zero).
- β = noise precision (large $\beta \Rightarrow$ low noise).

Posterior derivation

With N observations $\{(\mathbf{x}_i, y_i)\}$, define design matrix Φ with rows $\phi(\mathbf{x}_i)^T$.

Posterior (Gaussian $\times$ Gaussian = Gaussian):

$$p(\mathbf{w}|\mathcal{D}) = \mathcal{N}(\mathbf{m}_N, \mathbf{S}_N)$$

where:

$$\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi, \quad \mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{t}$$

- $\mathbf{m}_N$: posterior mean (regularized least squares solution).
- $\mathbf{S}_N$: posterior covariance (uncertainty in weights).

Predictive distribution

For a new input $\mathbf{x}_*$, integrate over weight uncertainty:

$$p(y_*|\mathbf{x}_*, \mathcal{D}) = \int p(y_*|\mathbf{x}_*, \mathbf{w}) p(\mathbf{w}|\mathcal{D}) d\mathbf{w}$$

Result (Gaussian):

$$p(y_*|\mathbf{x}_*, \mathcal{D}) = \mathcal{N}(\mathbf{m}_N^T \phi_*, \sigma_*^2)$$

where:

$$\sigma_*^2 = \underbrace{\beta^{-1}}_{\text{noise}} + \underbrace{\phi_*^T \mathbf{S}_N \phi_*}_{\text{weight uncertainty}}$$

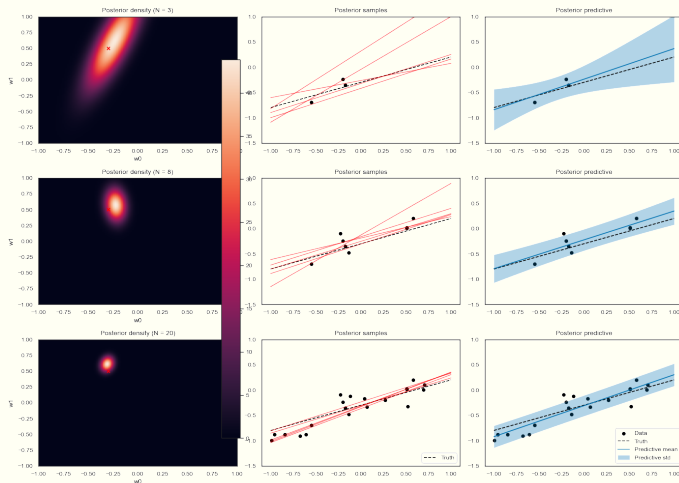
Two sources of uncertainty

$$\sigma_*^2 = \underbrace{\beta^{-1}}_{\text{aleatoric}} + \underbrace{\phi_*^T \mathbf{S}_N \phi_*}_{\text{epistemic}}$$

- **Aleatoric uncertainty** (β^{-1}): irreducible noise in the data. Cannot be reduced by collecting more data.
- **Epistemic uncertainty** ($\phi_*^T \mathbf{S}_N \phi_*$): uncertainty about model parameters. *Shrinks* as we collect more data.

This decomposition is central to Bayesian approaches!

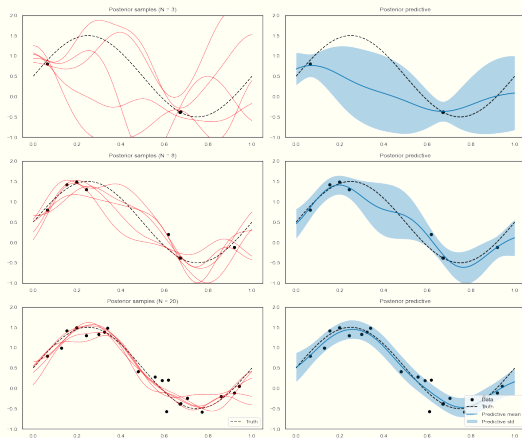
Posterior evolution with data



- **Left:** Posterior density over weights sharpens with more data.
- **Middle:** Posterior samples of functions become more consistent.
- **Right:** Predictive bands shrink where data exist.

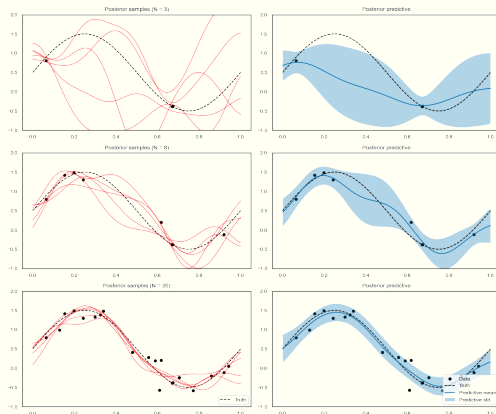
Nonlinear targets via basis functions

Replace identity basis with Gaussian basis: $\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2\sigma^2}\right)$



Linear model in ϕ space fits nonlinear targets in x space.

Key observation: uncertainty away from data



- Predictive bands are **tight** near observations.
- Bands **widen** away from data (epistemic uncertainty grows).
- This is exactly what we want for extrapolation warnings!

The model selection problem

How do we choose:

- Number of basis functions?
- Polynomial degree?
- Network architecture?

Cross-validation is one approach. Bayesian model selection uses the **evidence** (marginal likelihood):

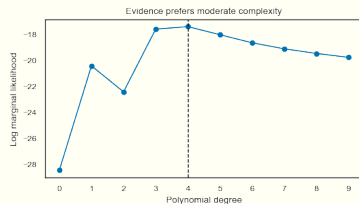
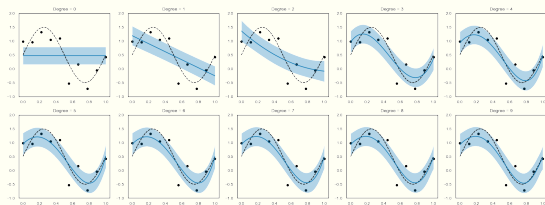
$$p(\mathcal{D}|\mathcal{M}) = \int p(\mathcal{D}|\theta, \mathcal{M}) p(\theta|\mathcal{M}) d\theta$$

Evidence as automatic Occam's razor

$$\ln p(\mathbf{t}|\alpha, \beta) = \underbrace{\frac{M}{2} \ln \alpha + \frac{N}{2} \ln \beta - E(\mathbf{m}_N)}_{\text{data fit}} - \underbrace{\frac{1}{2} \ln |\mathbf{S}_N^{-1}|}_{\text{complexity}} - \frac{N}{2} \ln(2\pi)$$

- Complex models can fit data well but have low prior probability for any specific fit.
- Simple models have high prior probability but may not fit data.
- Evidence balances automatically — no tuning needed.

Evidence in action: polynomial regression



- Peak at degree 3.
- Lower degrees: underfit.
- Higher degrees: overfit.

We can also optimize hyperparameters α, β by maximizing evidence:

$$\alpha^*, \beta^* = \arg \max_{\alpha, \beta} \ln p(\mathcal{D} | \alpha, \beta)$$

- Called “Empirical Bayes” or “Type II Maximum Likelihood.”
- Automatic regularization strength selection.
- No cross-validation needed.

The scalability problem

Bayesian linear regression works because:

- Gaussian prior $\times$ Gaussian likelihood = Gaussian posterior.

For neural networks:

- Posterior $p(\mathbf{w}|\mathcal{D})$ is **intractable** (no closed form).
- MCMC is too slow for millions of parameters.
- We need a fast approximation.

Variational inference: Approximate $p(\mathbf{w}|\mathcal{D})$ with a tractable $q_{\phi}(\mathbf{w})$.

The variational objective

Find $q_\phi(\mathbf{w})$ that is “close” to $p(\mathbf{w}|\mathcal{D})$ by minimizing KL divergence:

$$\text{KL}(q_\phi \| p(\cdot|\mathcal{D})) = \int q_\phi(\mathbf{w}) \ln \frac{q_\phi(\mathbf{w})}{p(\mathbf{w}|\mathcal{D})} d\mathbf{w}$$

Problem: This requires knowing $p(\mathbf{w}|\mathcal{D})$, which is intractable!

Solution: Rearrange to get the **Evidence Lower Bound (ELBO)**.

Deriving the ELBO

Start with log evidence:

$$\ln p(\mathcal{D}) = \ln \int p(\mathcal{D}, \mathbf{w}) d\mathbf{w}$$

Multiply and divide by $q_\phi(\mathbf{w})$, apply Jensen's inequality:

$$\ln p(\mathcal{D}) \geq \underbrace{\mathbb{E}_{q_\phi} [\ln p(\mathcal{D}|\mathbf{w})]}_{\text{expected log-likelihood}} - \underbrace{\text{KL}(q_\phi \| p(\mathbf{w}))}_{\text{complexity penalty}}$$

This is the **ELBO** $\mathcal{L}(\phi)$. Maximizing ELBO $\Leftrightarrow$ minimizing KL to posterior.

$$\mathcal{L}(\phi) = \underbrace{\mathbb{E}_{q_\phi}[\ln p(\mathcal{D}|\mathbf{w})]}_{\text{data fit}} - \underbrace{\text{KL}(q_\phi \| p(\mathbf{w}))}_{\text{stay close to prior}}$$

- **First term:** Encourages q_ϕ to put mass on weights that explain data.
- **Second term:** Penalizes q_ϕ for deviating from prior (regularization).

Same trade-off as MAP estimation, but for entire distributions!

The reparameterization trick

To optimize ELBO with gradient descent, we need $\nabla_{\phi} \mathbb{E}_{q_{\phi}}[f(\mathbf{w})]$.

Problem: Expectation depends on ϕ through the distribution.

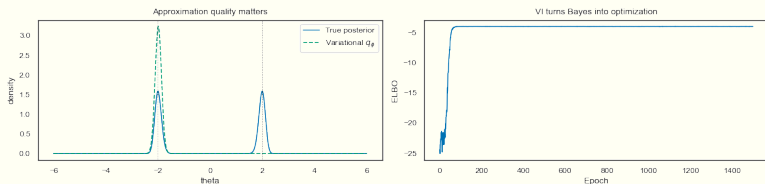
Solution: Reparameterize the sampling:

$$\mathbf{w} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Now $\mathbf{w}$ is a deterministic function of $\phi = \{\boldsymbol{\mu}, \boldsymbol{\sigma}\}$ and random $\boldsymbol{\epsilon}$:

$$\nabla_{\phi} \mathbb{E}_{q_{\phi}}[f(\mathbf{w})] = \mathbb{E}_{\boldsymbol{\epsilon}}[\nabla_{\phi} f(\boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon})]$$

VI example: approximating a bimodal posterior



- True posterior is bimodal (two plausible values of θ).
- Single Gaussian q_ϕ can only capture one mode.
- This is the **mean-field approximation** limitation.

Mean-field approximation

For computational tractability, we assume:

$$q_{\phi}(\mathbf{w}) = \prod_i q_{\phi_i}(w_i) = \prod_i \mathcal{N}(w_i | \mu_i, \sigma_i^2)$$

- Each weight has independent Gaussian posterior.
- Ignores correlations between weights.
- Trades accuracy for scalability.

Despite limitations, works surprisingly well in practice!

Apply variational inference to neural network weights:

- 1 Replace each weight w_{ij} with distribution $q(w_{ij}) = \mathcal{N}(\mu_{ij}, \sigma_{ij}^2)$.
- 2 Define prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \sigma_p^2 \mathbf{I})$.
- 3 Optimize ELBO via stochastic gradient descent.

This is **Bayes by Backprop** (Blundell et al., 2015).

Standard linear layer: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ with fixed $\mathbf{W}, \mathbf{b}$.

Bayesian linear layer:

- 1 Store $\mu_W, \rho_W, \mu_b, \rho_b$ (variational parameters).
- 2 Compute $\sigma = \text{softplus}(\rho) = \ln(1 + e^\rho)$ (ensures $\sigma > 0$).
- 3 Sample: $W = \mu_W + \sigma_W \odot \epsilon_W, \quad \epsilon_W \sim \mathcal{N}(0, 1)$.
- 4 Forward pass: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$.
- 5 Compute KL term for loss.

Bayes by Backprop loss

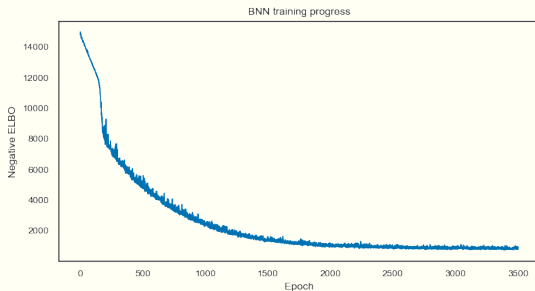
$$\mathcal{L} = \underbrace{\text{KL}(q_\phi(\mathbf{w}) \| p(\mathbf{w}))}_{\text{complexity cost}} - \underbrace{\mathbb{E}_{q_\phi} [\ln p(\mathcal{D} | \mathbf{w})]}_{\text{likelihood cost}}$$

Monte Carlo estimate with S weight samples:

$$\mathcal{L} \approx \frac{1}{S} \sum_{s=1}^S \left[\ln q_\phi(\mathbf{w}^{(s)}) - \ln p(\mathbf{w}^{(s)}) - \ln p(\mathcal{D} | \mathbf{w}^{(s)}) \right]$$

- Sample weights once (or few times) per minibatch.
- Backprop through sampled network as usual.
- Twice the parameters (mean + variance for each weight).

Training the BNN surrogate



- Architecture: $1 \rightarrow 64 \rightarrow 64 \rightarrow 1$ with tanh activations.
- Prior: $\mathcal{N}(0, 1)$ on all weights.
- Training: 3500 epochs, Adam optimizer, 6 MC samples per step.

Predictive distribution

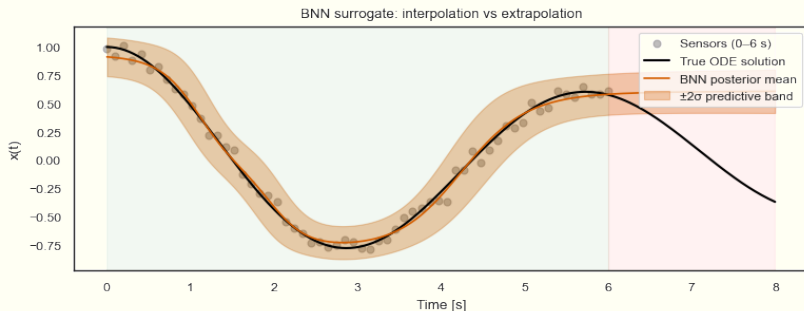
At test time, approximate predictive distribution via Monte Carlo:

$$p(y_*|x_*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S p(y_*|x_*, \mathbf{w}^{(s)}), \quad \mathbf{w}^{(s)} \sim q_\phi(\mathbf{w})$$

In practice:

- 1 Sample S weight configurations from q_ϕ .
- 2 Run forward pass for each to get S predictions.
- 3 Compute mean and variance of predictions.

BNN predictions with uncertainty



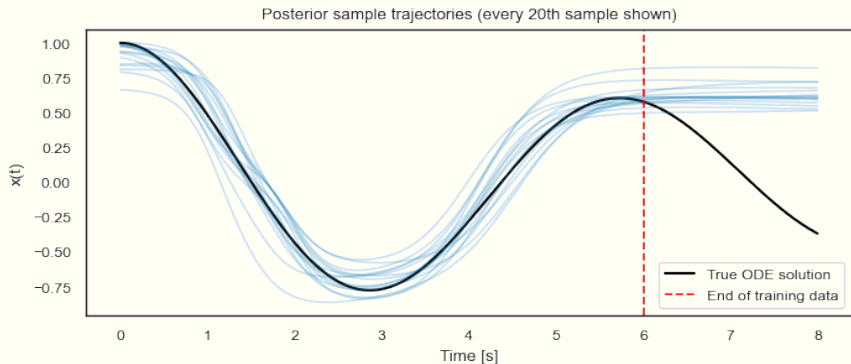
- **Green region** (0–6 s): Training data. Tight predictive bands.
- **Red region** (6–8 s): Extrapolation. Bands widen significantly.

Why uncertainty grows in extrapolation

- In training region: Many weight configurations fit the data $\Rightarrow$ predictions agree.
- In extrapolation: Weight configurations that agreed on training data **diverge** outside it.

This is exactly what we want! The model **knows what it doesn't know**.

Posterior samples tell the story



- Each curve: one forward pass with $\mathbf{w}^{(s)} \sim q_{\phi}(\mathbf{w})$.
- Curves cluster in training region, fan out when extrapolating.
- **Epistemic uncertainty** visualized directly.

Epistemic vs Aleatoric uncertainty revisited

- **Aleatoric** (data noise): Present everywhere, irreducible.
- **Epistemic** (model uncertainty):
 - Small where data exist (model is confident).
 - Large in extrapolation (model is uncertain).
 - Can be reduced with more data in those regions.

BNNs naturally capture both through the predictive distribution.

- **Computational cost:** $2\times$ parameters, multiple forward passes.
- **Mean-field approximation:** Ignores weight correlations.
- **Prior specification:** Results sensitive to prior choice.
- **Underestimated uncertainty:** VI tends to underestimate variance.

MC Dropout (Gal & Ghahramani, 2016):

- Keep dropout on at test time.
- Multiple forward passes $\approx$ posterior samples.
- No architecture changes needed!

Deep Ensembles (Lakshminarayanan et al., 2017):

- Train M networks with different initializations.
- Ensemble variance $\approx$ epistemic uncertainty.
- Often outperforms BNNs in practice.

When to use BNNs

Good fit for BNNs:

- Small to medium datasets where uncertainty matters.
- Safety-critical applications (medical, autonomous systems).
- Active learning (query where uncertainty is high).
- Scientific ML: need calibrated extrapolation warnings.

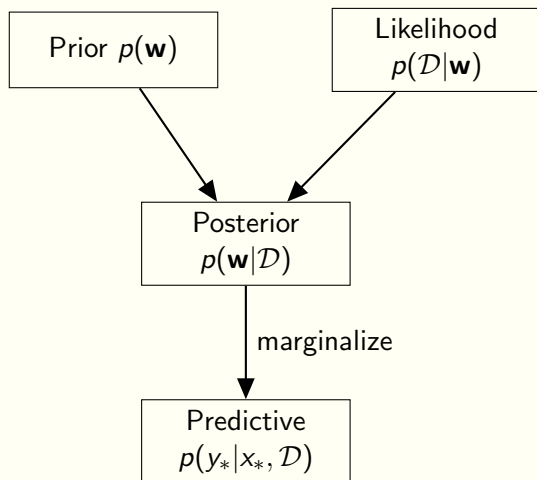
Consider alternatives:

- Large-scale vision/NLP (use ensembles or MC dropout).
- When computational budget is very limited.

Key takeaways

- 1 **Bayes' theorem** provides a principled framework for updating beliefs with data.
- 2 **Bayesian linear regression** gives exact posteriors — study it to build intuition.
- 3 **Evidence** automatically balances model fit vs complexity.
- 4 **Variational inference** turns Bayesian inference into optimization.
- 5 **BNNs** provide calibrated uncertainty, especially for extrapolation.

The big picture



BNNs approximate this pipeline with variational inference, giving us uncertainty-aware predictions for scientific machine learning.

- **Bishop, PRML Ch. 3:** Bayesian linear regression derivations.
- **Blundell et al. (2015):** “Weight Uncertainty in Neural Networks”
— Bayes by Backprop.
- **Gal & Ghahramani (2016):** “Dropout as a Bayesian Approximation.”
- **Lakshminarayanan et al. (2017):** “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles.”

Code: See accompanying notebook `09a-bnn.ipynb` for full implementation.